

NANDLand

Simulador de Circuitos Lógicos Universais

Disciplina: Programação A

O que você vai construir

Toda a computação moderna, do processador do seu celular a um supercomputador, pode ser inteiramente construída a partir de uma **única** primitiva: a porta lógica **NAND**. Esse resultado, chamado de *universalidade funcional*, garante que qualquer função booleana (AND, OR, NOT, XOR, somadores, memórias, decodificadores etc.) pode ser obtida combinando-se apenas NANDs.

Neste projeto você implementará, em Python, um **motor de simulação de circuitos digitais** fiel a esse princípio. Partindo apenas da **PortaNand** e do conceito de *fios* que transportam sinais booleanos, sua infraestrutura deverá ser capaz de:

- compor portas NAND para formar circuitos arbitrariamente complexos (NOT, AND, OR, XOR, somadores, registradores etc.);
- tratar cada circuito composto como se fosse uma nova porta de primeira classe, registrável em um **catálogo** reutilizável;
- simular o tempo de forma síncrona por meio de um *relógio* interno, permitindo que circuitos com **realimentação** (*latches*, *flip-flops*, registradores) funcionem corretamente sem recursão infinita;
- persistir o catálogo em arquivo, para que seus circuitos sobrevivam entre execuções;
- (opcional, bônus) ser operado por uma **interface gráfica** que apenas reflete o estado do motor, toda a lógica booleana mora no motor simulado.

O que você aprende. Três competências são exercitadas em paralelo:

1. **Orientação a Objetos:** cada conceito do domínio (pino, porta, circuito, catálogo, simulador) vira uma classe com responsabilidades estritas. Você é forçado a pensar em *polimorfismo*, não em verificações de tipo.
2. **Padrões de Projeto:** o motor é estruturado em torno de padrões clássicos. *Observer* para propagar sinais, *Composite* para criar circuitos complexos a partir da composição de circuitos simples, *Prototype* para realizar a clonagem, *Registry* para implementar um catálogo de circuitos, e *MVC* para implementar uma interface gráfica.
3. **Modelagem da realidade física:** você descobre, na prática, por que sistemas digitais reais usam *relógio* e *atraso de porta* em vez de propagação instantânea. A semântica adotada aqui é a mesma de simuladores profissionais (Verilog, VHDL).

Esqueleto fornecido: você não começa do zero

Atenção: o professor disponibiliza um **repositório-esqueleto** em Python contendo:

- a **estrutura completa de pacotes** do projeto;
- todas as **classes já declaradas** (`PinoEntrada`, `PinoSaida`, `ComponenteLogico`, `PortaNand`, `CircuitoComposto`, `CatalogoCircuitos`, `Simulador`, ...), com seus atributos, assinaturas de métodos, tipos e *docstrings*;
- a **bateria completa de testes unitários** (`pytest`) que serão usados na avaliação.

O corpo de cada método aparece marcado com `# TODO` (ou `raise NotImplementedError`). **Sua tarefa é apenas implementar esses corpos.** Não altere nomes de classes, nomes de pinos, assinaturas ou a organização dos módulos. Os testes automáticos dependem deles. Cada passo descrito nas fases a seguir corresponde a um conjunto identificável de `TODOs` no esqueleto. À medida que você os preenche, mais testes passam (`pytest -k fase1, fase2, ...`). Com a dinâmica de estudo do The Huxley você já deve estar acostumado com os testes. Para este projeto, cada fase tem uma bateria de testes que deve passar.

Regras obrigatórias

Para que o foco se mantenha na correta abstração orientada a objetos, as regras a seguir **não podem ser violadas** (a violação anula a respectiva fase):

- **Sem condicionais de tipo no motor.** É **proibido** usar `if/elif` ou `isinstance()` para checar o tipo de uma porta lógica ao avaliá-la. O comportamento deve ser puramente polimórfico.
- **GUI sem lógica booleana.** Se você fizer a Fase 5, a interface só captura eventos e exibe estados; nenhuma decisão lógica acontece nela (padrão MVC).
- **Persistência simples.** O catálogo deve ser salvo/carregado em arquivo textual simples (JSON sugerido, `pickle` aceito). **Não** use banco de dados, ORM ou frameworks de persistência. O motor deve funcionar sem o arquivo. Ele só preserva circuitos definidos pelo usuário.
- **NAND é a única primitiva.** `PortaNand` é o único componente lógico primitivo. Qualquer outra função (NOT, AND, OR, XOR, *latches*, *flip-flops*, registradores) deve **emergir** da composição de NANDs. Memória de estado vem de **realimentação**, não de um componente especial.
- **Simulação síncrona por ciclos.** A propagação *não* é instantânea. Cada `PinoSaida` tem buffer duplo (`valor_atual` e `valor_proximo`), e o tempo avança em fases alternadas de *avaliação* e *commit* controladas pelo `Simulador`. É isso que torna a realimentação possível sem recursão.

Roteiro de implementação (4 fases)

O projeto está dividido em quatro fases. **Não há prazos semanais fixos**: as fases podem ser executadas em qualquer ritmo, e ao final você deve entregar o máximo que conseguir concluir. Cada fase tem um **critério de aceitação** estrito que você deve validar antes de avançar.

Fase 1 — Sistema de Fiação Síncrono (*Observer* com Buffer Duplo)

Você construirá a infraestrutura de sinais com **simulação síncrona por ciclos**. Cada pino mantém dois valores ao mesmo tempo: o *valor atual* (lido pelos componentes durante o ciclo) e o *valor próximo* (escrito durante o ciclo, ainda invisível). O avanço do tempo é controlado pelo Simulador.

Modelo conceitual de um ciclo (tick). A Figura 2 resume o ciclo; a Figura 1 mostra o comportamento temporal de um PinoSaida; a Figura 3 adianta por que esse mecanismo é essencial para os circuitos sequenciais da Fase 4.

- Fase de Avaliação:** cada componente lê o *valor atual* das suas entradas e escreve o *valor próximo* nas suas saídas. Nada disso é visível para outros componentes ainda.
- Fase de Commit:** o simulador percorre todos os pinos de saída e copia *valor próximo* → *valor atual* simultaneamente. As entradas conectadas são notificadas aqui (não na escrita).

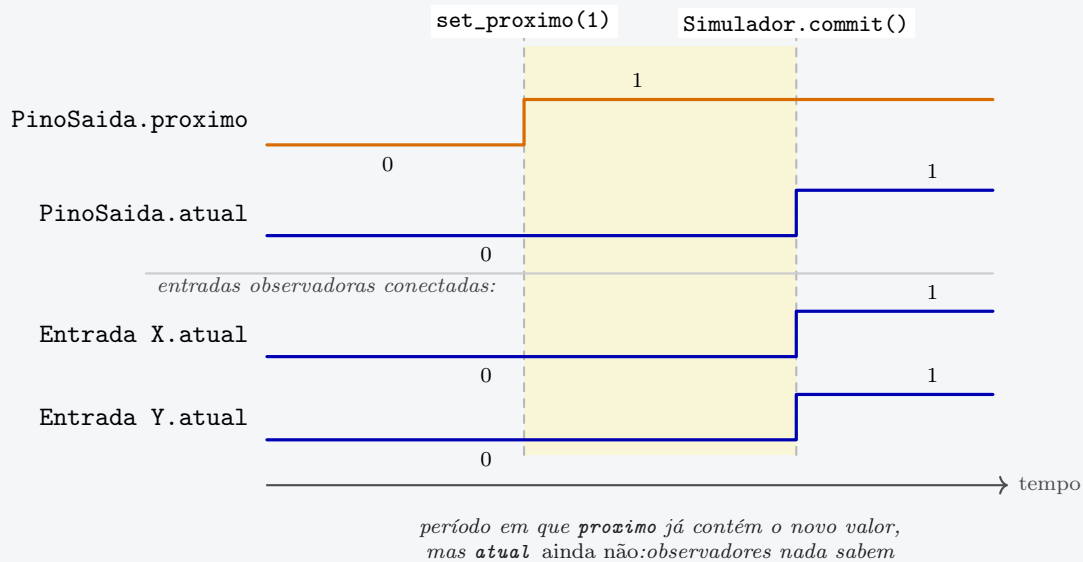


Figura 1: Comportamento temporal de um PinoSaida com dois observadores. A trilha laranja é `valor_proximo`, que reage imediatamente a `set_proximo(1)`. As trilhas azuis são `valor_atual` no pino e nas PinoEntrada conectadas: todas ficam em 0 durante a faixa amarela e saltam para 1 *simultaneamente* quando o `Simulador.commit()` promove `proximo` → `atual` e dispara as notificações.

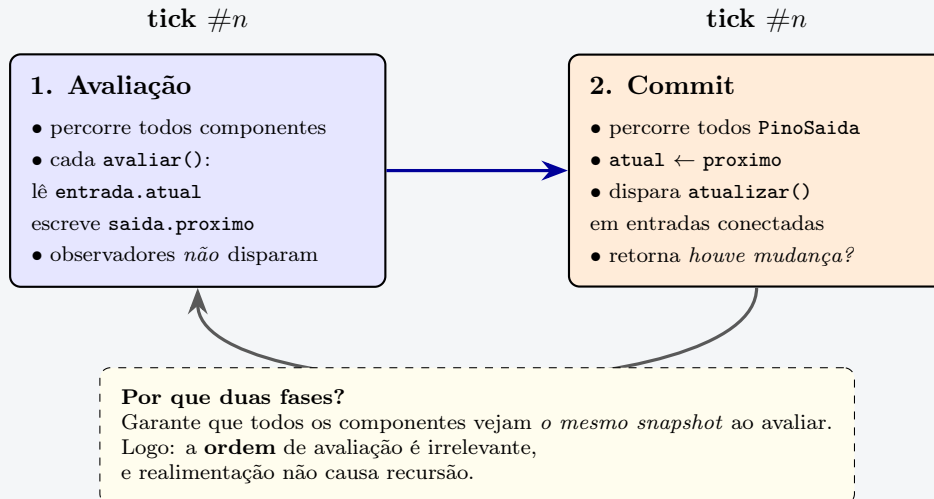


Figura 2: Ciclo do `Simulador.tick().run_until_stable()` repete o ciclo até que a fase de commit não detecte mais mudanças (ou atinja o limite de ticks, indicando oscilação).

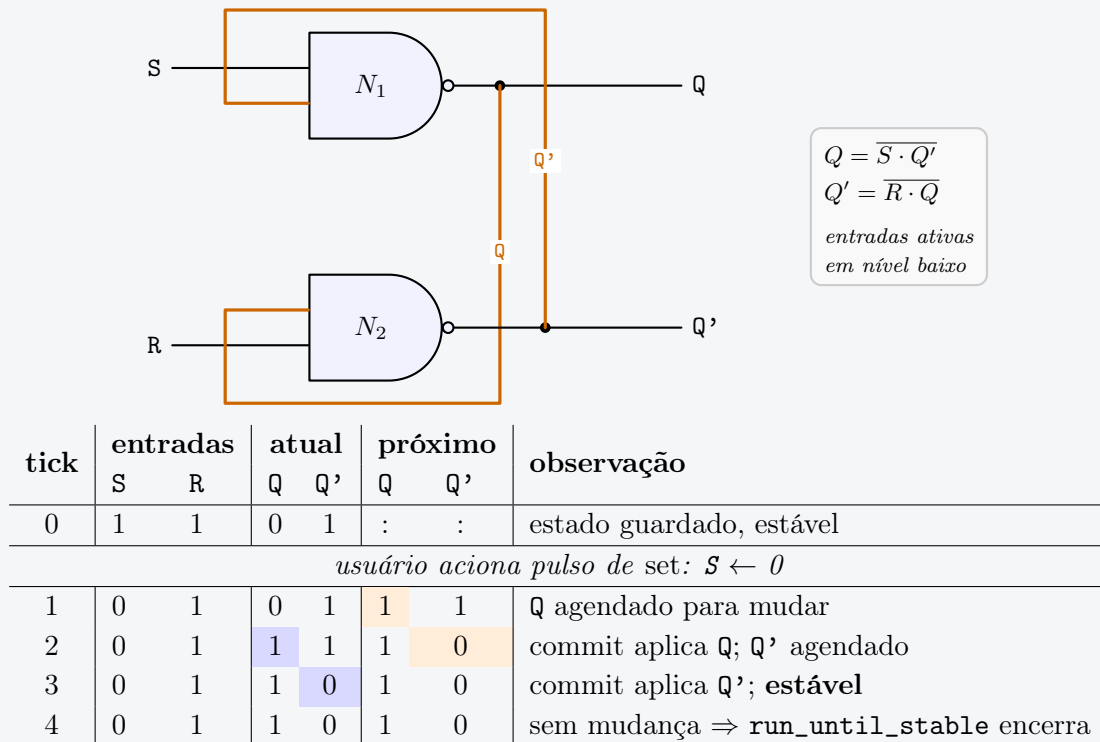


Figura 3: SR-latch construído com duas `PortaNand` cruzadas (entradas ativas em nível baixo). Sem o buffer duplo, a primeira escrita em Q já notificaria N_2 , que escreveria em Q', que notificaria $N_1 \dots$ provocando recursão infinita.

O que implementar.

- **Passo 1.1:Pino de entrada.** Implemente os `TODOs` da classe responsável por receber sinais. Ela deve guardar um único valor booleano (o *valor atual* lido pelos componentes) e oferecer uma forma de ser atualizada externamente. Esse mesmo ponto de escrita é usado tanto pelo simulador, na fase de *commit*, quanto por sinais externos do usuário/GUI: o buffer duplo é responsabilidade exclusiva do pino de saída.

- **Passo 1.2:Pino de saída com buffer duplo.** Preencha os TODOs da classe que carrega *dois* valores simultâneos: o que está visível ao restante do circuito no ciclo corrente e o que está agendado para o próximo ciclo. Ela também precisa manter uma lista de observadores (os pinos de entrada conectados). A escrita no valor agendado **não** pode notificar observadores; a notificação só acontece quando o ciclo é fechado, no momento em que o valor agendado é promovido a valor visível.
- **Passo 1.3:Simulador.** Implemente os TODOs da classe que orquestra o tempo. Ela mantém o conjunto de componentes em jogo e o conjunto de pinos de saída cujos buffers precisam ser fechados a cada ciclo. Um ciclo (*tick*) consiste nas duas fases descritas acima:avaliação e *commit*:e deve indicar se algum sinal efetivamente mudou. Além do ciclo unitário, ofereça uma operação que repita ciclos até que o circuito se estabilize, com um teto de iterações para detectar osciladores.

Critério de aceitação

Script de teste com um PinoSaida S conectado a duas PinoEntrada X e Y (Figura 1). Após `S.set_proximo(True)`, **antes** do `commit` ambas as entradas devem ler **False**; **após** o `commit` (ou `tick()`), ambas devem ler **True** *simultaneamente*. Mostre também que escritas concorrentes no mesmo ciclo (`set_proximo(True)` seguido de `set_proximo(False)` antes do `commit`) só preservam o último valor, sem notificações intermediárias.

Fase 2 — Componente Base e Porta Raiz (*Composite* parte 1)

Aqui você define a primeira e *única* porta lógica real do sistema.

- **Passo 2.1:Componente abstrato.** Complete a classe-base que define o contrato comum a tudo que pode ser avaliado pelo simulador: avaliar-se, clonar-se e expor recursivamente seus pinos de saída internos (este último é o gancho que o simulador usa ao registrar um componente).
- **Passo 2.2:Porta NAND.** Implemente os TODOs da única porta primitiva do sistema. Ela é uma especialização do componente abstrato, com duas entradas e uma saída próprias.
- **Passo 2.3:Avaliação.** Preencha a avaliação da NAND segundo a sua definição lógica, respeitando o contrato do buffer duplo: a avaliação *lê* apenas o valor visível das entradas e *agenda* o resultado na saída. É **proibido** mexer no valor atual diretamente ou disparar qualquer propagação durante a avaliação.
- **Passo 2.4:Clonagem.** A clonagem deve produzir uma nova porta independente, com pinos próprios e sem reaproveitar conexões da original. Esse comportamento é o que o catálogo da Fase 3 vai explorar.
- **Passo 2.5:Exposição de saídas.** Implemente o gancho que devolve a(s) saída(s) interna(s) da porta para o simulador.

Ordem sugerida de implementação. As Fases 1 e 2 devem ser feitas em conjunto porque o `Simulador` só pode ser testado com algum `ComponenteLogico` concreto. Implemente nesta ordem: pinos → `ComponenteLogico` → `PortaNand` → `Simulador`, validando a Fase 1 já com uma `PortaNand` no circuito de teste.

Critério de aceitação

Teste unitário validando a tabela-verdade completa da `PortaNand` usando o `Simulador`: para cada combinação de entradas, basta *um único* `tick()` para que a saída reflita o resultado esperado.

Fase 3 — Circuitos Compostos e Catálogo (*Composite, Prototype, Registry*)

A fase central do projeto: fazer com que circuitos complexos se comportem exatamente como se fossem uma única porta lógica.

- **Passo 3.1:Circuito composto.** Implemente os `TODOs` da classe que representa um circuito formado pela composição de outros componentes. Ela é, ela própria, um componente lógico: sua avaliação delega às avaliações dos subcomponentes, e a exposição de saídas internas concatena recursivamente as saídas de todos eles. Externamente, o circuito precisa expor um conjunto nomeado de pinos de entrada e de pinos de saída, que funcionam como a “borda” visível para quem o usa.

Fan-out de entradas externas. Um pino externo de entrada pode precisar alimentar *vários* pinos internos: na porta `NOT`, por exemplo, o pino externo de entrada alimenta as duas entradas da única `NAND`. Para isso, há uma especialização do pino de entrada com a noção de *espelhos*: ao ser atualizada, ela escreve no próprio valor e repassa o sinal a cada pino interno espelhado. Como ainda é um pino de entrada comum do ponto de vista do tipo, qualquer saída pode conectar-se a ela uniformemente. Preencha os `TODOs` correspondentes.

- **Passo 3.2:Catálogo de circuitos.** Implemente os `TODOs` do *registry* de protótipos. Ele já nasce com a `NAND` registrada, e deve permitir registrar novos protótipos, listá-los e instanciá-los: sempre devolvendo um clone independente do protótipo, jamais a referência original.
- **Passo 3.3:Clonagem de compostos.** Complete a clonagem do circuito composto de modo que toda a sua estrutura interna (subcomponentes, pinos e conexões) seja duplicada sem compartilhar referências com o original.
- **Passo 3.4:Persistência do catálogo.** Implemente os `TODOs` de salvar e carregar o catálogo em arquivo textual simples. Cada circuito persistido precisa descrever seu nome, seus subcomponentes (referenciados pelo nome de catálogo), as conexões internas entre eles e o mapeamento dos pinos externos para os pinos internos. A primitiva `NAND` **não** é serializada: ela é sempre recriada em código. Apenas circuitos derivados, definidos pelo usuário, são persistidos. O carregamento deve respeitar a ordem de dependências entre circuitos (ordenação topológica).

Critério de aceitação

Crie programaticamente uma porta `NOT` (uma `PortaNand` com as duas entradas conectadas ao mesmo sinal) dentro de um `CircuitoComposto`, registre-a no catálogo, e prove que modificações estruturais ou de estado em uma instância clonada **não** afetam o protótipo nem outras instâncias. Toda avaliação deve passar pelo `Simulador`. Em seguida: `salvar("catalogo.json")`, criar um novo `CatalogoCircuitos` vazio, `carregar("catalogo.json")` e verificar que a porta `NOT` reconstruída produz a mesma tabela-verdade.

Fase 4 — Lógica Sequencial por Realimentação (Memória Emergente)

Agora você colhe o fruto da simulação síncrona da Fase 1: construir memória de estado **só com NANDs**, sem nenhum componente primitivo novo. A memória surge naturalmente da realimentação cruzada entre saídas e entradas, viabilizada pelo atraso de um ciclo do buffer duplo.

Por que funciona. Num SR-latch com duas NANDs e saídas cruzadas, se a propagação fosse instantânea o sistema entraria em recursão infinita ou divergiria. Com a simulação por ciclos, cada NAND avalia com base no *valor atual* do ciclo anterior; o `commit` aplica todas as saídas em paralelo. Após poucos `ticks` o circuito estabiliza num dos dois estados estáveis do biestável: exatamente o comportamento físico real. (O estado meta-estável só ocorre quando `S = R = False` simultaneamente.)

- **Passo 4.1:SR-latch.** Construa um circuito composto correspondente a um SR-latch montado com duas NANDs cujas saídas se alimentam reciprocamente, e registre-o no catálogo com o nome previsto pelo esqueleto. Suas entradas externas são ativas em nível baixo; as duas saídas externas são o estado armazenado e seu complemento. **Atenção à inicialização:** logo após a montagem, o circuito deve ser colocado em modo *hold* (ambas as entradas em nível alto); do contrário, partindo de pinos zerados, ele entraria no estado indefinido e poderia oscilar.
- **Passo 4.2:D-latch.** Construa o D-latch, que acrescenta ao SR-latch um *enable* e um sinal de dado. Quando o *enable* está ativo, a saída segue o dado; quando inativo, a saída retém o último valor. Toda a lógica deve sair da composição de NANDs: não introduza novas primitivas. O esqueleto indica o nome do circuito e quais pinos externos expor.
- **Passo 4.3:Flip-flop D (*master-slave*).** Construa o flip-flop D encadeando dois D-latches em série, de modo que o *enable* de um receba o clock externo e o do outro receba o clock invertido (obtido por composição de NANDs). O efeito desejado é que a saída só mude nas **bordas de subida** do clock.
- **Passo 4.4:Registrador de 1 bit.** Encapsule o flip-flop D em um circuito composto que exponha as entradas de dado e clock e a saída armazenada, com o nome previsto pelo esqueleto.

Cuidados com estabilização.

- Componentes sequenciais podem levar mais de um `tick` para estabilizar (tipicamente 2–4 ciclos para o SR-latch, mais para o flip-flop). Sempre chame `run_until_stable()` após mudar um sinal externo.
- Um SR-latch com `S = R = False` é **indefinido** fisicamente e pode oscilar no simulador. O limite `max_ticks` de `run_until_stable` deve sinalizar essa condição (exceção ou status apropriado).

Critério de aceitação

Teste integrado que:

- (i) instancia um `REGISTRO_1BIT` via catálogo;
- (ii) aplica `D = True`, dispara uma borda de subida em `CLK`, chama `run_until_stable` e verifica `Q = True`;
- (iii) muda `D = False` sem acionar o clock, executa novos ticks e verifica que `Q` permanece `True` (retenção de estado);
- (iv) dispara nova borda de subida em `CLK` e verifica `Q = False`.

Demonstrando, assim, que memória de estado emerge *puramente* da composição de NANDs sob simulação síncrona.

Fase 5 (Extra / Opcional) — Interface Gráfica (MVC) Entrega facultativa, conta como bônus

Não é obrigatória. As Fases 1–4 já constituem um simulador funcional, operável via scripts. A Fase 5 é um desafio extra para grupos que terminarem as fases obrigatórias e quiserem pontuação adicional.

Construa a camada visual com a **framework de GUI de sua escolha** (Tkinter, CustomTkinter, PySide/PyQt, Kivy, pygame, web via Flask/FastAPI + HTML, ou qualquer outra). A escolha é livre, desde que a interface respeite o modelo síncrono: nenhuma interação atualiza estado interno diretamente: tudo passa pelo **Simulador**.

- **Painel de Catálogo:** lista/menu com os itens disponíveis no `CatalogoCircuitos` (incluindo `SR_LATCH`, `D_FLIP_FLOP`, `REGISTRO_1BIT`).
- **Área de Instanciação:** criação de componentes na tela e ferramentas para conectar saídas a entradas. Cada conexão na GUI vira um `PinoSaida.conectar(PinoEntrada)` no modelo.
- **Controle de Simulação:** botões `Step` (um tick), `Run` (`run_until_stable`) e `Clock` (alterna um pino de clock visível). Opcionalmente, mostre o número do ciclo atual.
- **Persistência:** menu com `Salvar Catálogo` / `Carregar Catálogo` delegando aos métodos do `CatalogoCircuitos`. Se existir um `catalogo.json` padrão no diretório, carregue-o automaticamente ao iniciar.
- **Interação:** botões/chaves para injetar sinais (0/1) e LEDs virtuais que leem `valor_atual` após cada tick.

Critério de aceitação

Critério final (bônus). Pela GUI:

- (a) criar, instanciar e registrar uma porta **AND** (NAND seguida de NOT — duas NANDs no total) e demonstrar sua tabela-verdade clicando em chaves visuais;
- (b) instanciar um `REGISTRO_1BIT`, injetar um valor em `D`, pulsar `Clock` e observar que o LED de `Q` retém o valor mesmo após mudanças subsequentes em `D` sem novo pulso de clock.

Entrega e avaliação

Formato de entrega. Repositório Git público (ou `.zip`) com código-fonte (a partir do esqueleto fornecido, com os TODOs preenchidos), os testes `pytest` passando e um `README` com instruções de

execução.

Distribuição de pontos (sugerida). Fases 1+2: 30% Fase 3: 35% Fase 4: 35% Fase 5 (extra): até +20% de bônus.

Grupos. Mínimo de 2 e máximo de 4 integrantes. Todos devem ter commits significativos no repositório. **Entregas individuais sofrerão desconto de 50% da nota final:** o projeto foi dimensionado para colaboração e exercita explicitamente divisão de tarefas e revisão por pares (*pull requests*).

Como sua nota individual é calculada. A nota de cada aluno **não** é simplesmente a do grupo. Ela combina:

1. **Prova individual presencial (60%).** Sem consulta, sobre os conceitos do projeto: padrões aplicados (*Observer*, *Composite*, *Prototype*, *Registry*, MVC), semântica da simulação síncrona, polimorfismo e justificativa das decisões arquiteturais. Você deve ser capaz de explicar e modificar qualquer parte do código entregue pelo seu grupo.
2. **Nota do grupo (25%).** Produto final: testes passando, qualidade do código, aderência às regras (sem *isinstance*, sem lógica na GUI, NAND como única primitiva), clareza da documentação. Esta parcela é modulada pelo seu fator de participação f_i .
3. **Fator de participação individual (15%), auditado via GitHub.** Cada aluno recebe $f_i \in [0,5; 1,2]$ calculado a partir do histórico do repositório: número de *commits* qualificados (excluindo *lock files*, binários e código gerado), linhas adicionadas/removidas no núcleo, distribuição temporal dos commits (concentrar tudo na última semana penaliza), *pull requests* abertos, revisões em PRs de colegas, *issues* fechadas e detecção de anomalias (commits gigantes, mensagens genéricas, autoria divergente, *force-push* reescrevendo histórico alheio). A normalização é feita *dentro de cada grupo*.

$$\text{nota_final}_i = 0,60 \cdot \text{prova}_i + 0,25 \cdot (\text{nota_grupo} \cdot f_i) + 0,15 \cdot (f_i \cdot 10)$$

Regras de auditoria do GitHub.

- Configure `git config user.email` com seu e-mail institucional. Commits com autoria não verificável podem não ser contabilizados.
- É **vedado** usar `git commit --author="Outro <...>"` para atribuir trabalho a colegas: isso é fraude acadêmica.
- Toda mudança na *main* deve passar por *pull request* revisado por outro integrante do grupo.
- *Force-push* em *main* e **commits desproporcionalmente grandes**: definidos como aqueles que (a) adicionam mais de **300 linhas líquidas** no núcleo (excluindo arquivos gerados, *lock files* e binários) ou (b) concentram mais de **25% do total de linhas adicionadas pelo autor** em todo o projeto: são sinalizados como anomalias e exigem justificativa.

Salvaguardas anti-carona.

- Se $\text{prova}_i < 4,0$, a nota final é igual à da prova: não há aprovação por contribuição de grupo sem domínio individual demonstrado.
- Discrepância maior que 2 pontos entre a prova e o fator de participação ($|\text{prova}_i - 10f_i| > 2$) implica **entrevista oral** de defesa do código, na qual você deverá explicar trechos centrais que afirma ter escrito. A entrevista pode ajustar tanto f_i quanto a nota da prova.

Ao final do projeto você receberá o detalhamento numérico do seu f_i , as métricas que o compõem e a lista de eventuais anomalias detectadas em seu histórico de commits.

Resumo do que entregar.

1. Repositório com o esqueleto preenchido (Fases 1–4 obrigatórias; Fase 5 opcional).
2. `pytest` verde no diretório-raiz do projeto.
3. `README.md` com instruções de instalação e execução, e a lista dos integrantes do grupo.
4. Histórico de commits saudável (vide regras de auditoria).

Bom projeto! Ao final, você terá um simulador capaz de modelar desde uma simples porta AND até, em princípio, um pequeno processador: comprovando, na prática, o caminho que vai de um único transistor lógico até uma CPU completa.